5A:

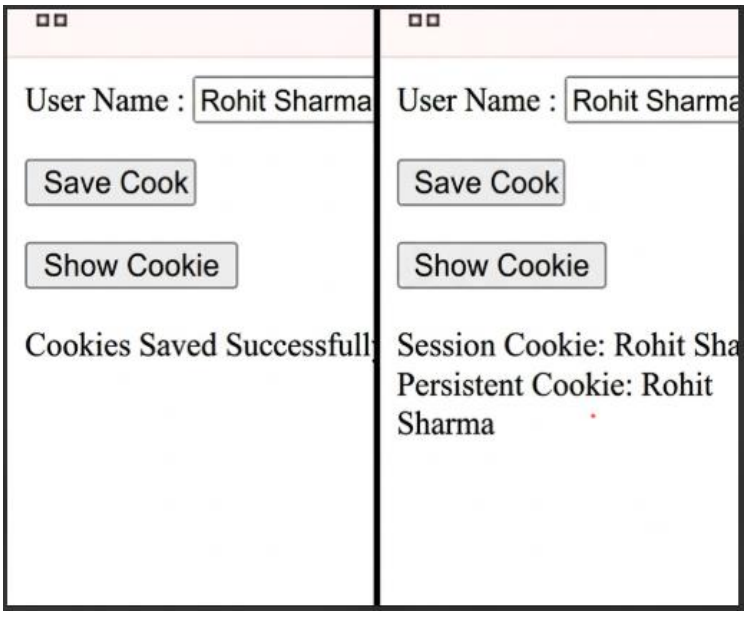
AIM: Create a web application to demonstrate the use of different types of cookies.

PROGRAM CODE:

```
protected void btnSave_Click(object sender, EventArgs e)
{
    HttpCookie sessionCookie = new HttpCookie("SessionUser");
    sessionCookie.Value = txtName.Text;
    Response.Cookies.Add(sessionCookie);
    HttpCookie persistentCookie = new HttpCookie("PersistentUser");
    persistentCookie.Value = txtName.Text;
    persistentCookie.Expires = DateTime.Now.AddDays(7);
    Response.Cookies.Add(persistentCookie);
    lblOutput.Text = "Cookies Saved Successfully";
}
protected void btnShow_Click(object sender, EventArgs e)
{    string sessionData = Request.Cookies["SessionUser"] != null
        ? Request.Cookies["SessionUser"].Value
        : "Not Found";

    string persistentData = Request.Cookies["PersistentUser"] != null
        ? Request.Cookies["PersistentUser"].Value
        : "Not Found";
    lblOutput.Text = "Session Cookie: " + sessionData + "<br/>Persistent Cookie: " + persistentData;
}
```

OUTPUT:



5B:

AIM: Create a product Table in Built in SQL Server of Visual Studio and perform Insert Operation using Entity Framework Core.

PROGRAM CODE:

C#

```
namespace ProductEFCore.Models
{
    public class Products
    {
        public int Id { get; set; }
        public required string ProductName { get; set; }
        public double Price { get; set; }
    }
}
```

AppDbContext.cs *(inside Data folder)*

C#

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;

namespace ProductEFCore.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Products> Products { get; set; }
    }
}
```

Program.cs

C#

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Data;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews(); // or AddControllers() depending on your project type
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection"),
        sqlOptions => sqlOptions.EnableRetryOnFailure()
    ));

var app = builder.Build();
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
```

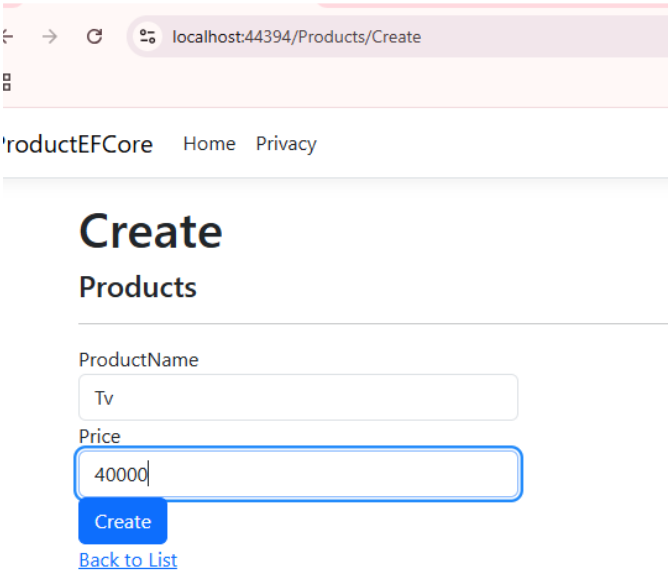
```
app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

OUTPUT:



5C:

AIM: Develop a web application to perform CRUD Operations(Create, Read, Update, Delete) on Employee table using EF Core.

PROGRAM CODE:

Employee.cs

C#

```
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public double Salary { get; set; }
    }
}
```

AppDbContext.cs

C#

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
```

```
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

appsettings.json

JSON

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True
",
    "AppDbContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true
"
  }
}
```

Program.cs

C#

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

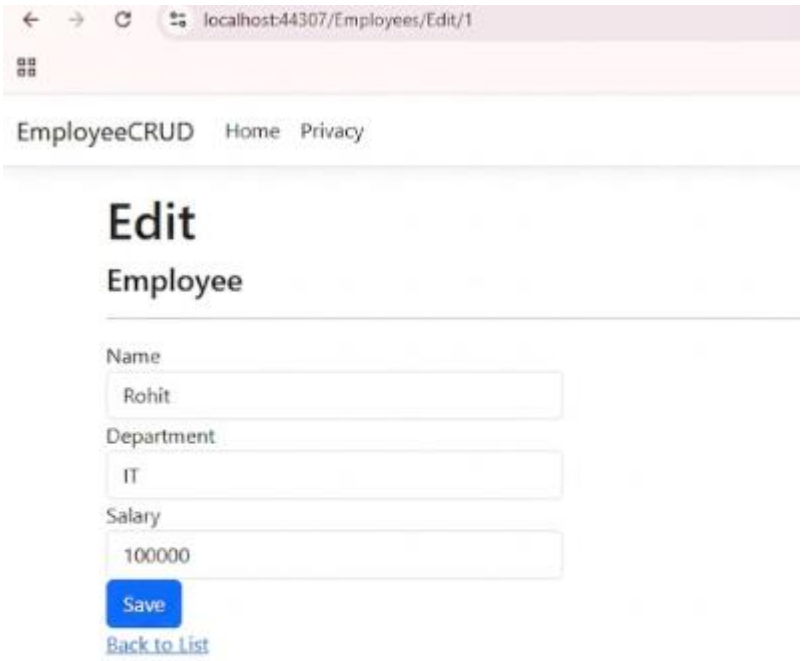
app.UseRouting();
```

```
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

OUTPUT:





5D:

AIM: Authentication and authorization in ASP.NET Core.

PROGRAM CODE:

HomeController.cs

C#

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [Authorize]
        public IActionResult Secret()
        {
            return View();
        }
    }
}
```

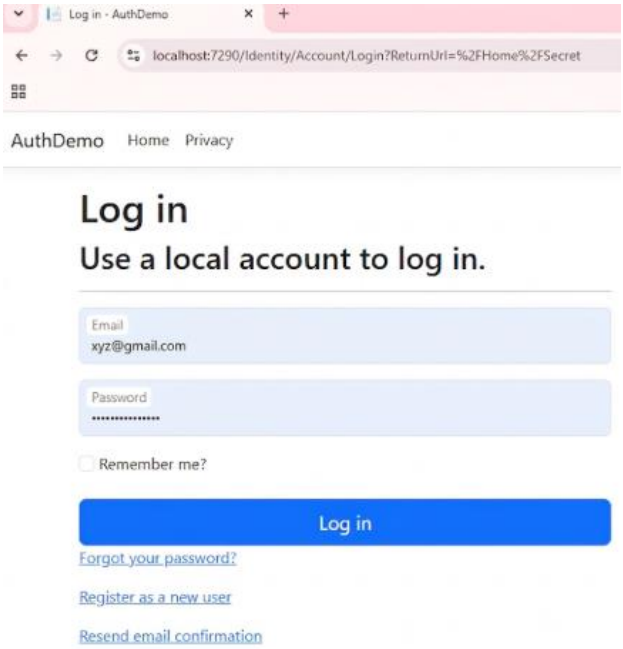
Views/Home/Secret.cshtml

HTML

```
@{
    ViewData["Title"] = "Secret Page";
}

<h2>Authorized Users Only</h2>
```

OUTPUT:



10A:

AIM: Create a PWA with a web App Manifest file including:

- App name
- Icons
- Start URL
- Display mode
- Implement Push Notifications in a PWA.

PROGRAM CODE:

manifest.json

JSON

```
{
  "name": "My PWA Application",
  "short_name": "MyPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

index.html

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>My PWA</title>
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">
</head>
<body>
    <h1>My Progressive Web App</h1>
    <button onclick="showNotification()">
        Show Notification
    </button>
    <script src="app.js"></script>
</body>
</html>
```

wwwroot/app.js

JavaScript

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
```

```
}

function showNotification() {
    Notification.requestPermission().then(permission => {
        if (permission === 'granted') {
            navigator.serviceWorker.ready.then(registration => {
                registration.showNotification('PWA Notification', {
                    body: 'Push Notification Working Successfully!',
                    icon: 'icons/icon1.png'
                });
            });
        }
    });
}
```

wwwroot/service-worker.js

JavaScript

```
self.addEventListener('install', event => {
    console.log('Service Worker Installed');
});

self.addEventListener('activate', event => {
    console.log('Service Worker Activated');
});
```

Program.cs

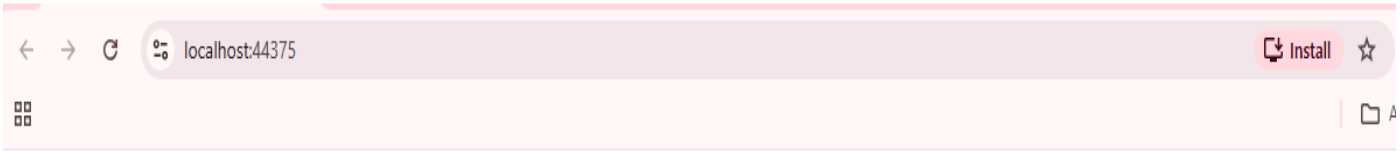
C#

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

OUTPUT:



My Progressive Web App

Show Notification

10B:

AIM: E-Commerce PWA with Product Listing and Offline Cart

PROGRAM CODE:

manifest.json

JSON

```
{
  "name": "E-Commerce PWA",
  "short_name": "ShopPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

index.html

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>E-Commerce PWA</title>
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">
    <style>
        body {
            font-family: Arial;
            margin: 20px;
        }
        .container {
            display: flex;
            gap: 30px;
        }

        .cart-section {
            width: 30%;
            border: 1px solid gray;
            padding: 15px;
            min-height: 300px;
        }
        .product-section {
            width: 70%;
        }
        .products {
            display: flex;
```

```
        flex-wrap: wrap;
        gap: 15px;
      }
      .product {
        border: 1px solid gray;
        padding: 10px;
        width: 200px;
      }
      button {
        padding: 5px 10px;
        cursor: pointer;
      }
    </style>
</head>
<body>
    <h1>E-Commerce PWA</h1>
    <div class="container">
      <div class="cart-section">
        <h2>Cart Items</h2>
        <ul id="cartList"></ul>
      </div>
      <div class="product-section">
        <h2>Products</h2>
        <div class="products">
          <div class="product">
            <h3>Laptop</h3>
            <p>Price: ₹ 50000</p>
            <button onclick="addToCart('Laptop')">Add to Cart</button>
          </div>
          <div class="product">
            <h3>Mobile</h3>
            <p>Price: ₹ 20000</p>
            <button onclick="addToCart('Mobile')">Add to Cart</button>
          </div>
          <div class="product">
            <h3>Headphones</h3>
            <p>Price: ₹ 3000</p>
            <button onclick="addToCart('Headphones')">Add to Cart</button>
          </div>
        </div>
      </div>
    </div>
    <script src="app.js"></script>
</body>
</html>
```

wwwroot/app.js

JavaScript

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
      .then(() => console.log('Service Worker Registered'));
}
let cart = JSON.parse(localStorage.getItem('cart')) || [];
displayCart();
function addToCart(product) {
    cart.push(product);
    localStorage.setItem('cart', JSON.stringify(cart));
    displayCart();
}
```

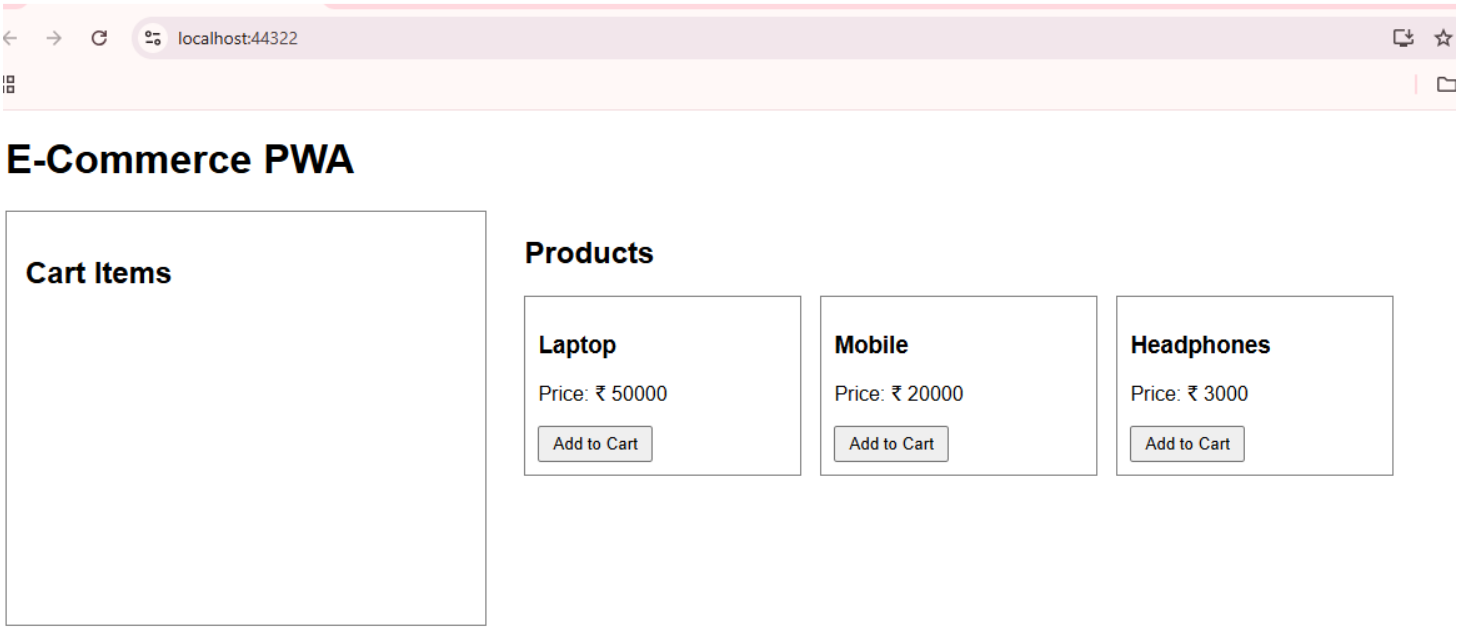
```
function displayCart() {
    let cartList = document.getElementById('cartList');
    cartList.innerHTML = "";
    cart.forEach(item => {
        let li = document.createElement('li');
        li.textContent = item;
        cartList.appendChild(li);
    });
}
```

wwwroot/service-worker.js

JavaScript

```
const CACHE_NAME = 'ecommerce-cache-v1';
const urlsToCache = [
    '/',
    '/index.html',
    '/app.js',
    '/manifest.json'
];
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});
self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

OUTPUT:



PRACTICAL 1A:

AIM: Create an MVC Application to perform basic Arithmetic Operations.

PROGRAM CODE :

Model File (Models/Calculator.cs)

C#

```
namespace ArithmeticMVC.Models
{
    public class Calculator
    {
        public double Number1 { get; set; }
        public double Number2 { get; set; }
        public required string Operation { get; set; }
        public double Result { get; set; }
    }
}
```

Controller File (Controllers/HomeController.cs)

C#

```
using Microsoft.AspNetCore.Mvc;
using ArithmeticMVC.Models;
namespace ArithmeticMVC.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Index(Calculator calc)
        {
            switch (calc.Operation)
            {
                case "+":
                    calc.Result = calc.Number1 + calc.Number2;
                    break;
                case "-":
                    calc.Result = calc.Number1 - calc.Number2;
                    break;
                case "*":
                    calc.Result = calc.Number1 * calc.Number2;
                    break;
                case "/":
                    if (calc.Number2 != 0)
                    {
                        calc.Result = calc.Number1 / calc.Number2;
                    }
                    break;
            }
            return View(calc);
        }
```

```
    }
}
```

Index View File (Views/Home/Index.cshtml)

HTML

```
@model ArithmeticMVC.Models.Calculator

<h2>Arithmetic Operations</h2>

<form method="post">
    <label>Enter Number1</label><br />
    <input type="number" step="any" asp-for="Number1" />
    <br /><br />

    <label>Enter Number2</label><br />
    <input type="number" step="any" asp-for="Number2" />
    <br /><br />

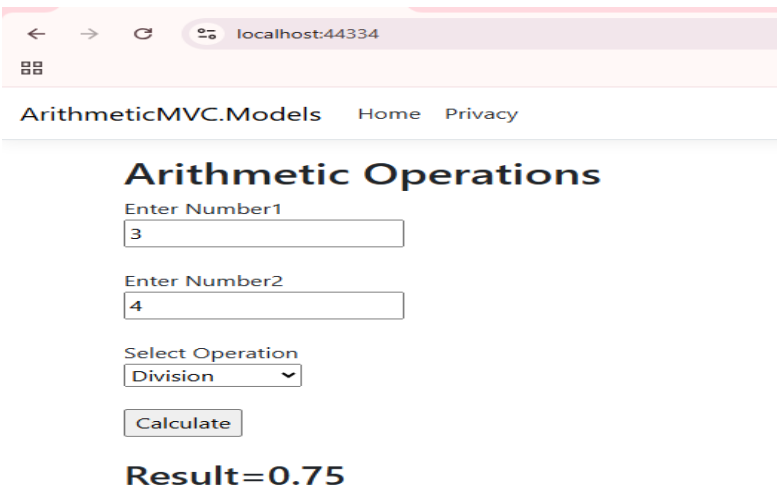
    <label>Select Operation</label><br />
    <select asp-for="Operation">
        <option value="+">Addition</option>
        <option value="-">Subtraction</option>
        <option value="*">Multiplication</option>
        <option value="/">Division</option>
    </select>
    <br /><br />

    <input type="submit" value="Calculate" />
</form>

<br />

@if (Model != null)
{
    <h3>Result=@Model.Result</h3>
}
```

OUTPUT:



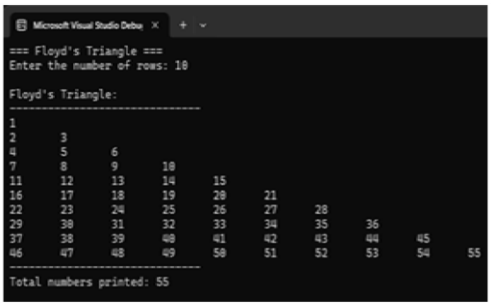
1B:

AIM: To write a C# program to print Floyd's triangle up to N rows, where N is input by the user.

PROGRAM CODE :

```
using System; namespace FloydTriangle
{     class Program
   {         static void Main(string[] args)
      {
          Console.WriteLine("=== Floyd's Triangle ===");
Console.Write("Enter the number of rows: ");            int n =
int.Parse(Console.ReadLine());            int number = 1;
          Console.WriteLine("\nFloyd's                              Triangle:");
Console.WriteLine(new string('-', 30));            for (int i = 1; i <= n; i++)
          {               for (int j = 1; j <= i; j++)
             {
                Console.Write(number + "\t");                  number++;
             }
             Console.WriteLine();
          }
          Console.WriteLine(new string('-', 30));
          Console.WriteLine($"Total numbers printed: {number - 1}");
          Console.ReadKey();
      }
   }
}
```

OUTPUT :



PRACTICAL 1C:

AIM:

To write a C# program that accepts names of 10 cities from the user and displays them in sorted (alphabetical) order.

PROGRAM CODE :

```
using System; namespace SortCities

{     class Program

   {         static void Main(string[] args)

      {

         string[] cities = new string[10];

         Console.WriteLine("=== City Name Sorter ===");

         Console.WriteLine("Enter names of 10 cities:");
Console.WriteLine(new string('-', 30));                    for (int i = 0; i < 10; i++)

         {

            Console.Write($"City {i + 1}: ");              cities[i] =
Console.ReadLine();

         }

         Array.Sort(cities, StringComparer.OrdinalIgnoreCase);
Console.WriteLine("\n--- Cities in Sorted Order ---");          for (int i = 0; i <
cities.Length; i++)

         {

            Console.WriteLine($"{i + 1}. {cities[i]}");

         }
         Console.WriteLine(new string('-', 30));

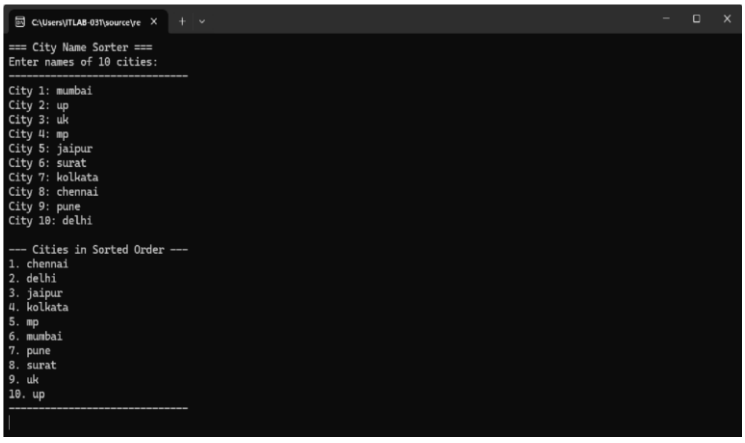
         Console.ReadKey();

      }

   }

}
```

OUTPUT :



PRACTICAL 2:

2A: FUNCTION OVERLOADING

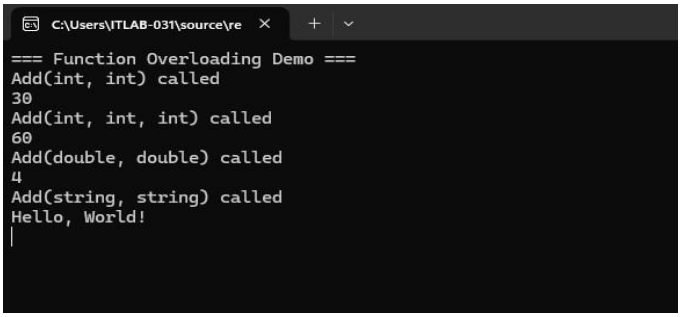
AIM: To demonstrate function overloading in C# by creating multiple methods with the same name but different parameters.

PROGRAM CODE :

```
using System; namespace FunctionOverloading
{     class Calculator
   {                public int Add(int a, int b)
      {
         Console.WriteLine("Add(int, int) called");             return a + b;
      }         public int Add(int a, int b, int c)
      {
         Console.WriteLine("Add(int, int, int) called");             return a + b + c;
      }
      public double Add(double a, double b)
      {
         Console.WriteLine("Add(double, double) called");             return a + b;
      }
      public string Add(string a, string b)
      {
         Console.WriteLine("Add(string, string) called");             return a + b;
      }
   }
   class Program
   {         static void Main(string[] args)
      {
         Calculator calc = new Calculator();
         Console.WriteLine("=== Function Overloading Demo ===");
         Console.WriteLine("Result: " + calc.Add(10, 20));
         Console.WriteLine("Result: " + calc.Add(10, 20, 30));
         Console.WriteLine("Result: " + calc.Add(1.5, 2.5));
         Console.WriteLine("Result: " + calc.Add("Hello, ", "World!"));
         Console.ReadKey();
      }
```

```
    }
}
```

OUTPUT :



2B:

AIM: To demonstrate all types of inheritance in C#: Single, Multilevel, Hierarchical, and Multiple Inheritance (via Interfaces).

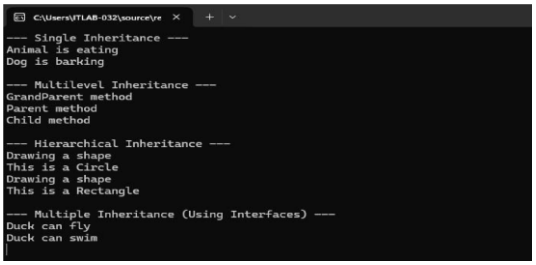
PROGRAM CODE :

```
using System; namespace InheritanceDemo
{    class Animal
   {        public void Eat()
      {
         Console.WriteLine("Animal is eating");
      }
   }
   class Dog : Animal
   {        public void Bark()
      {
         Console.WriteLine("Dog is barking");
      }
   }
   class GrandParent
   {
      public void Method1()
      {
         Console.WriteLine("GrandParent method");
      }
   }
   class Parent : GrandParent
   {
```

```
    public void Method2()
    {
        Console.WriteLine("Parent method");
    }
}     class Child : Parent
{
    public void Method3()
    {
        Console.WriteLine("Child method");
    }
}
class Shape
{
    public void Draw()
    {
        Console.WriteLine("Drawing a shape");
    }
}     class Circle : Shape
{         public void Info()
    {
        Console.WriteLine("This is a Circle");
    }
}
class Rectangle : Shape
{         public void Info()
    {
        Console.WriteLine("This is a Rectangle");
    }
}     interface IFlyable
{         void Fly();
}
interface ISwimmable
{         void Swim();
}
```

```
class Duck : IFlyable, ISwimmable
{       public void Fly()
    {
        Console.WriteLine("Duck can fly");
    }
    public void Swim()
    {
        Console.WriteLine("Duck can swim");
    }
}
class Program
{       static void Main(string[] args)
    {
        Console.WriteLine("=== Inheritance Demo ===\n");
        Console.WriteLine("--- Single Inheritance ---");          Dog d = new
Dog();          d.Eat();
        d.Bark();
        Console.WriteLine("\n--- Multilevel Inheritance ---");          Child c = new
Child();          c.Method1();
        c.Method2();
        c.Method3();
        Console.WriteLine("\n--- Hierarchical Inheritance ---");          Circle cir = new
Circle();          cir.Draw();          cir.Info();
        Rectangle rec = new Rectangle();          rec.Draw();
rec.Info();
        Console.WriteLine("\n--- Multiple Inheritance (Interfaces) ---");          Duck duck = new
Duck();          duck.Fly();          duck.Swim();
        Console.ReadKey();
    }
}
}
```

OUTPUT:



2C:

AIM: To demonstrate constructor overloading in C# by defining multiple constructors with different parameter lists in the same class.

PROGRAM CODE :

```
using System; namespace ConstructorOverloading

{     class Student

   {        public string Name;        public int
Age;        public string Course;        public
Student()

      {

         Name = "Unknown";

         Age = 0;

         Course = "N/A";

         Console.WriteLine("Default constructor called");

      }

      public Student(string name)

      {

         Name = name;

         Age = 18;

         Course = "BSc IT";

         Console.WriteLine("1-parameter constructor called");

      }

      public Student(string name, int age)
      {

         Name = name;

         Age = age;

         Course = "BSc IT";

         Console.WriteLine("2-parameter constructor called");

      }

      public Student(string name, int age, string course)

      {

         Name = name;

         Age = age;

         Course = course;

         Console.WriteLine("3-parameter constructor called");
```

```
    }
    public void Display()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}, Course: {Course}");
    }
}
class Program
{       static void Main(string[] args)
    {
        Console.WriteLine("=== Constructor Overloading Demo ===\n");          Student s1 = new Student();          s1.Display();          Console.WriteLine();

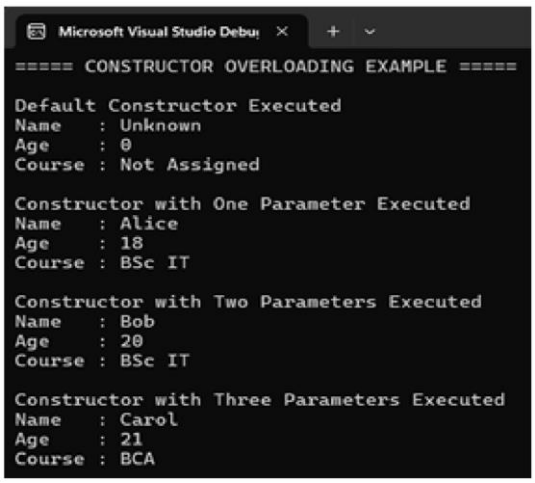
        Student s2 = new Student("Alice");
        s2.Display();          Console.WriteLine();

        Student s3 = new Student("Bob", 20);          s3.Display();          Console.WriteLine();

        Student s4 = new Student("Carol", 21, "BCA");          s4.Display();

        Console.ReadKey();
    }
}
}
```

OUTPUT:



2D:

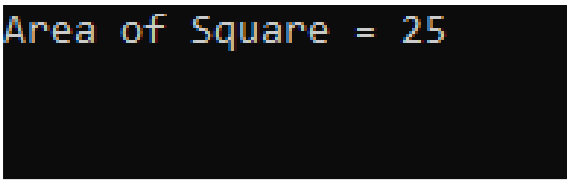
AIM: Write a program to use Interfaces feature of C#

PROGRAM CODE:

```
using System;
```

```
interface IShape
{
   void Area();
}
class Square : IShape
{
   int side = 5;
   public void Area()
   {
      Console.WriteLine("Area of Square = " + (side * side));
   }
}
class Program
{
   static void Main()
   {
      Square s = new Square();
      s.Area();
      Console.ReadLine();
   }
}
```

OUTPUT:



2E:

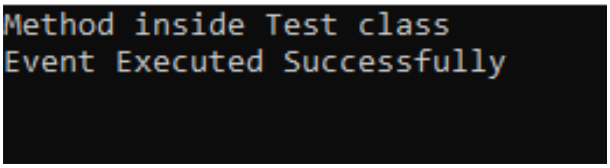
AIM: Write a program to use Delegates and events feature of C#

PROGRAM CODE :

```
using System;
public delegate void Message();
class Test
{
   public event Message Notify;
   public void Show()
   {
```

```
        Console.WriteLine("Method inside Test class");
        Notify();
    }
}
class Program
{
    public static void Display()
    {
        Console.WriteLine("Event Executed Successfully");
    }
    static void Main()
    {
        Test t = new Test();
        t.Notify += Display;
        t.Show();
        Console.ReadLine();
    }
}
```

OUTPUT:



2F:

AIM: Write a program to use Exception handling feature of C#

PROGRAM CODE :

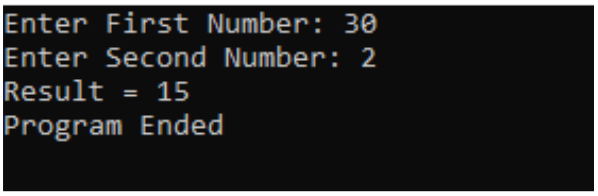
```
using System;
class Program
{
    static void Main()
    {
        int a, b, result;

        try
        {
            Console.Write("Enter First Number: ");
            a = Convert.ToInt32(Console.ReadLine());
            Console.Write("Enter Second Number: ");
            b = Convert.ToInt32(Console.ReadLine());
```

```
            result = a / b;
            Console.WriteLine("Result = " + result);
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Cannot divide by zero");
        }
        catch (FormatException)
        {
            Console.WriteLine("Invalid Input");
        }
        finally
        {
            Console.WriteLine("Program Ended");
        }

        Console.ReadLine();
    }
}
```

OUTPUT:



PRACTICAL 3A:

AIM: Create methods add(), multiply(),subtract(),divide() with suitable parameters and call these methods using concept of C# delegate.

PROGRAM CODE :

```
using System;

public delegate void Calculation(int a, int b);

class Arithmetic

{

   public void Add(int a, int b)

   {

      Console.WriteLine("Addition = " + (a + b));

   }

   public void Subtract(int a, int b)

   {

      Console.WriteLine("Subtraction = " + (a - b));

   }

   public void Multiply(int a, int b)

   {

      Console.WriteLine("Multiplication = " + (a * b));

   }

   public void Divide(int a, int b)

   {

      Console.WriteLine("Division = " + (a / b));

   }

}

class Program

{

   static void Main()

   {

      Arithmetic obj = new Arithmetic();

      Calculation c;

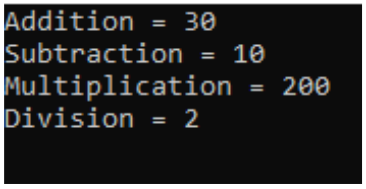
      c = obj.Add;

      c(20, 10);

      c = obj.Subtract;
```

```
        c(20, 10);
        c = obj.Multiply;
        c(20, 10);
        c = obj.Divide;
        c(20, 10);
        Console.ReadLine();
    }
}
```

OUTPUT:



3B:

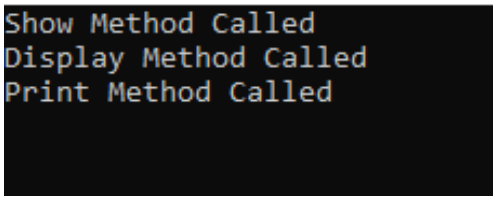
AIM: Write a program using multicast delegate.

PROGRAM CODE :

```
using System;
public delegate void Demo();
class Test
{
    public void Show()
    {
        Console.WriteLine("Show Method Called");
    }
    public void Display()
    {
        Console.WriteLine("Display Method Called");
    }
    public void Print()
    {
        Console.WriteLine("Print Method Called");
    }
}
```

```
class Program
{
    static void Main()
    {
        Test t = new Test();
        Demo d;
        d = t.Show;
        d += t.Display;
        d += t.Print;
        d();
        Console.ReadLine();
    }
}
```

OUTPUT:



3C:

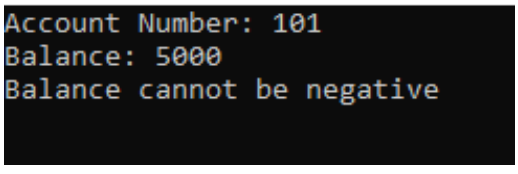
AIM: Create a class Bank Account with Account Number and Balance. Implement property for Balance with validation (Balance cannot be negative).

PROGRAM CODE :

```
using System;
class BankAccount
{
    private double balance;
    public double Balance
    {
        get
        {
            return balance;
        }
        set
```

```
        {
            if (value >= 0)
            {
                balance = value;
            }
            else
            {
                Console.WriteLine("Balance cannot be negative");
            }
        }
    }
    public int AccountNumber { get; set; }
}
class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();
        b.AccountNumber = 101;
        b.Balance = 5000;
        Console.WriteLine("Account Number: " +
b.AccountNumber);
        Console.WriteLine("Balance: " + b.Balance);
        b.Balance = -2000;
        Console.ReadLine();
    }
}
```

OUTPUT:



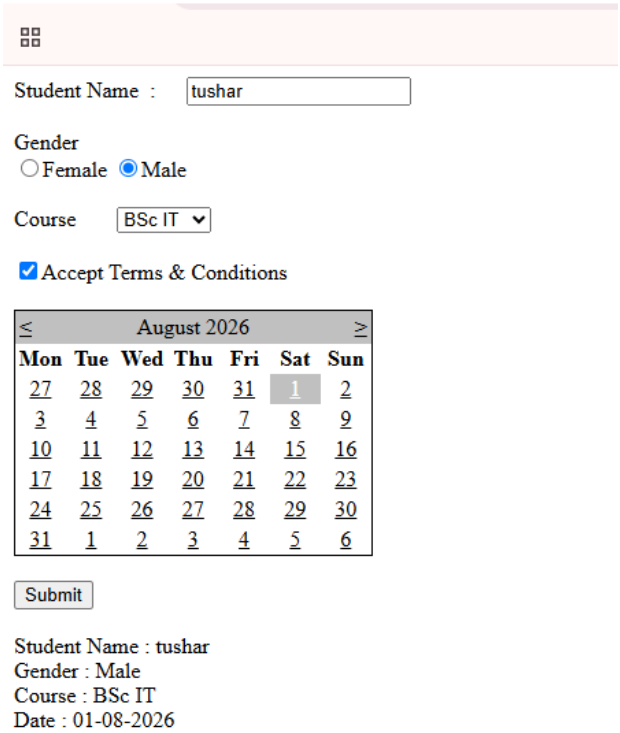
PRACTICAL 4A:

AIM: Create an ASP.NET Web Form application using simple server controls.

PROGRAM CODE :

```
Protected Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles btnSubmit.Click
    Dim gender As String = ""
    If RbMale.Checked Then
        gender = "Male"
    ElseIf rbFemale.Checked Then
        gender = "Female"
    End If
    If chkTerms.Checked Then
        lblOutput.Text = "Student Name : " & txtName.Text &
            "<br/>Gender : " & gender &
            "<br/>Course : " & ddlCourse.SelectedValue &
            "<br/>Date : " & calDate.SelectedDate.ToShortDateString()
    Else
        lblOutput.Text = "Please accept Terms & Conditions"
    End If
End Sub
```

OUTPUT:



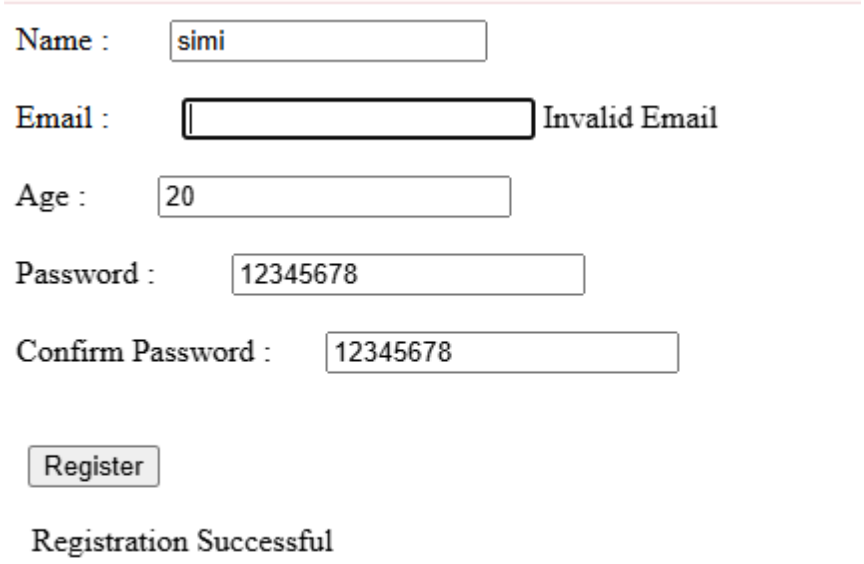
4B:

AIM: Create a Registration form to demonstrate use of various validation controls.

PROGRAM CODE :

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (Page.IsValid)
    {
        lblOutput.Text = "Registration Successful";
    }
}
```

OUTPUT:



4C:

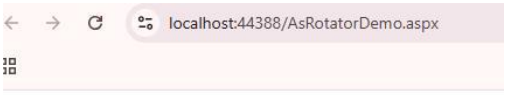
AIM: Create a Web Form to demonstrate use of Adrotator Control.

PROGRAM CODE :

```
<Advertisements>
  <Ad>
    <ImageUrl>Images/ad1.jpg</ImageUrl>
    <NavigateUrl>https://www.google.com/</NavigateUrl>
    <AlternateText>Google Ad</AlternateText>
    <Impressions>50</Impressions>
  </Ad>
  <Ad>
    <ImageUrl>Images/ad2.jpg</ImageUrl>
    <NavigateUrl>https://www.microsoft.com/</NavigateUrl>
```

```
    <AlternateText>Microsoft Ad</AlternateText>
    <Impressions>50</Impressions>
  </Ad>
</Advertisements>
```

OUTPUT:





PRACTICAL 6A:

AIM: Create a simple website for a college Information Portal and convert it into a progressive web application.

PROGRAM CODE :

HTML:

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>Welcome to xyz College</h3>
    <p>Courses Offered:</p>
    <ul>
        <li>BSC IT</li>
        <li>BSC CS</li>
        <li>BCA</li>
    </ul>
    <p>Contact: 9818763456</p>
</div>
```

Service Worker File (service-worker.js)

JavaScript

```
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});

self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
```

JSON

```
{
  "name": "College Information Portal",
  "short_name": "CollegePortal",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
```

```
    "sizes": "512x512",
    "type": "image/png"
   }
  ]
}
```

Manifest Link Tag (_Layout.cshtml - inside <head>)

HTML

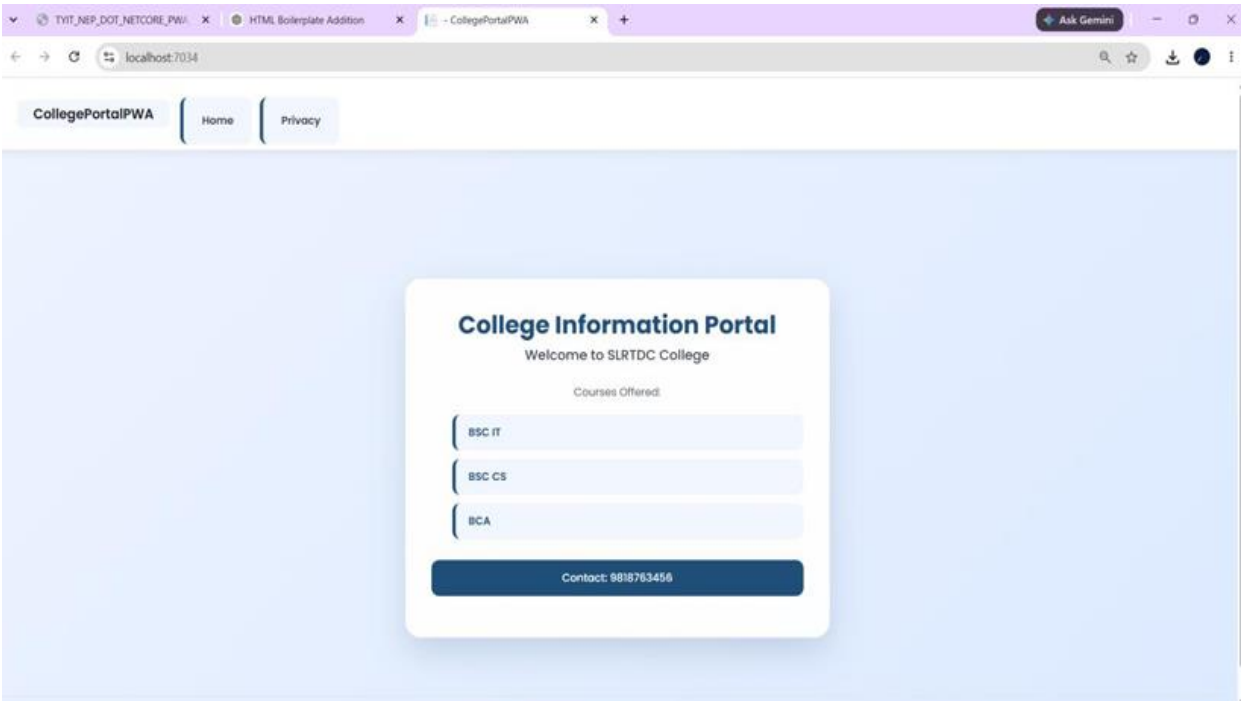
```
<link rel="manifest" href="~/manifest.json">
```

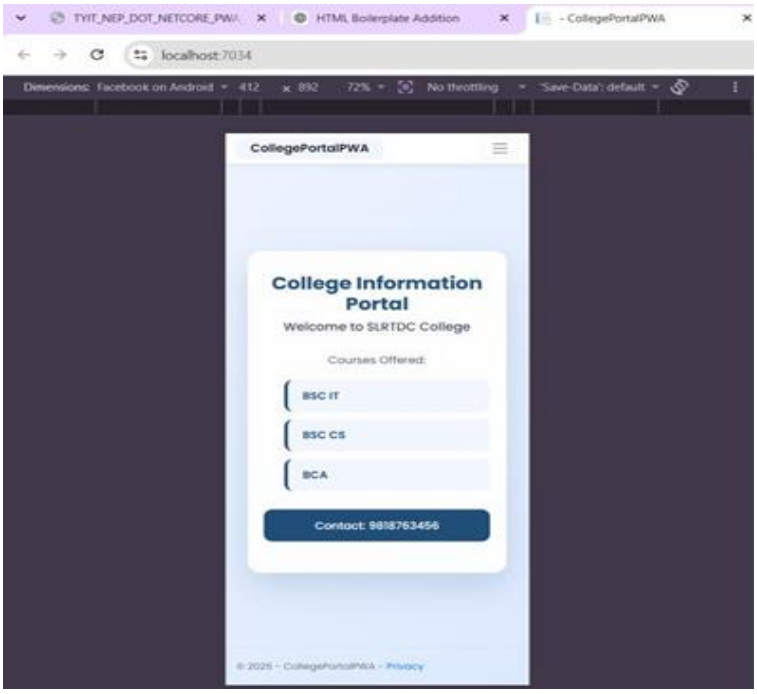
Service Worker Registration Script (_Layout.cshtml - before </body>)

HTML

```
<script>
   if ('serviceWorker' in navigator) {
      navigator.serviceWorker.register('/service-worker.js')
         .then((registration) => {
            console.log('Service Worker Registered!', registration);
         })
         .catch((error) => {
            console.log('Service Worker Registration Failed:', error);
         });
   }
</script>
```

OUTPUT:





6B:

AIM: Create a PWA that displays a custom offline page when the user is not connected to the internet.

PROGRAM CODE :

HTML

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>Welcome to XYZ College</h3>
    <p>This website supports Offline Mode.</p>
</div>
```

Offline Fallback Page (offline.html)

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline</title>
</head>
<body>
    <h1>No Internet Connection</h1>
    <h3>Please check your internet connection.</h3>
</body>
</html>
```

Manifest File (manifest.json)

JSON

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
```

```
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Service Worker File (service-worker.js)

JavaScript

```
const CACHE_NAME = "offline-cache-v1";
const urlsToCache = [
    "/",
    "/offline.html"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request)
            .catch(() => {
                return caches.match("/offline.html");
            })
    );
});
```

Manifest Link Tag (_Layout.cshtml - inside <head>)

HTML

```
<link rel="manifest" href="~/manifest.json">
```

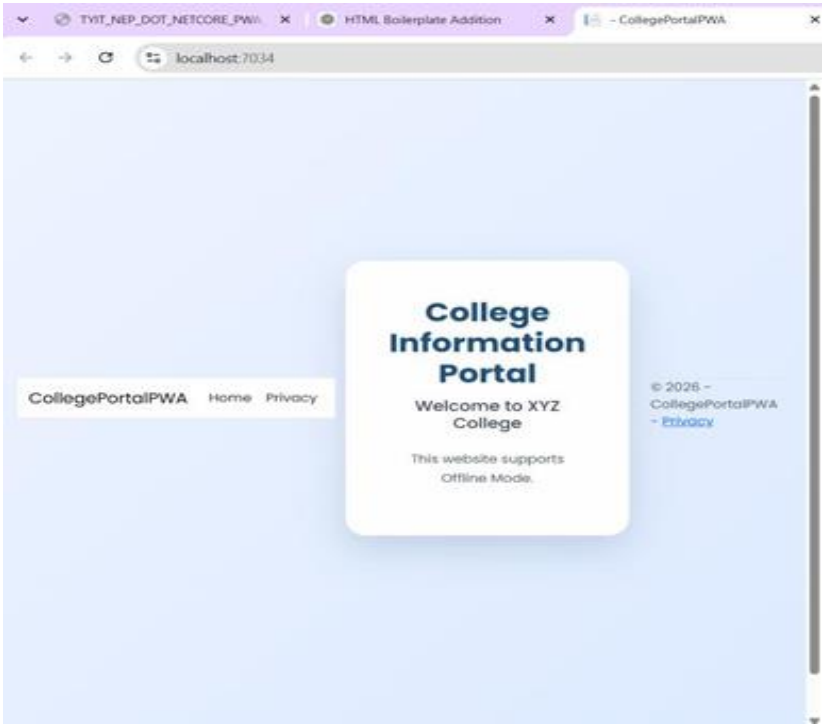
Service Worker Registration Script (_Layout.cshtml - before </body>)

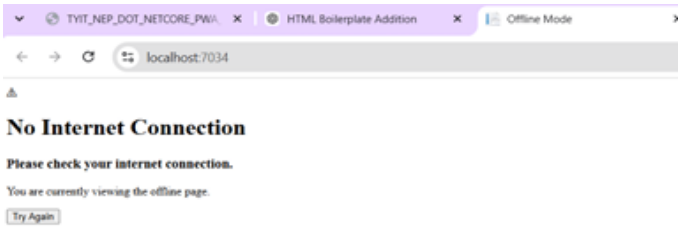
HTML

```
<script>
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/service-worker.js')
```

```
            .then(() => console.log('Service Worker Registered'));
    }
</script>
```

OUTPUT:





PRACTICAL 7A:

AIM: Develop a PWA that works in offline mode using service Worker.

PROGRAM CODE :

HTML

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>PWA Offline Demo</h3>
    <p>This application works without internet connection.</p>
</div>
```

Manifest File (manifest.json)

JSON

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Service Worker File (service-worker.js)

JavaScript

```
const CACHE_NAME = "pwa-cache-v1";
const urlsToCache = [
    "/",
    "/Home/Index"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});

self.addEventListener("fetch", event => {
```

```
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

Manifest Link Tag (_Layout.cshtml - inside <head>)

HTML

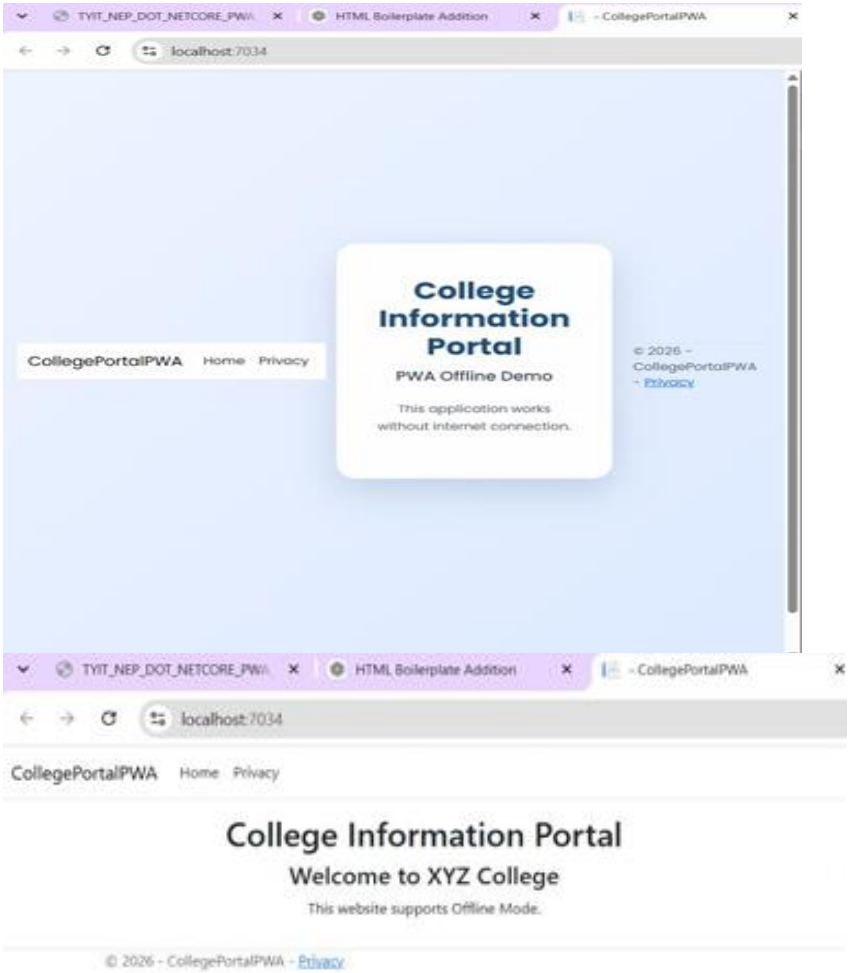
```
<link rel="manifest" href="/manifest.json">
```

Service Worker Registration Script (_Layout.cshtml - before </body>)

HTML

```
<script>
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/service-worker.js')
            .then(() => console.log('Service Worker Registered'));
    }
</script>
```

OUTPUT:



7A:

AIM: Create a Student Record Management PWA.

PROGRAM CODE :

Model File (Models/Student.cs)

C#

```
namespace StudentRecordPWA.Models
{
   public class Student
   {
      public int Id { get; set; }
      public string Name { get; set; }
      public string Course { get; set; }
      public int Age { get; set; }
   }
}
```

Controller File (Controllers/StudentsController.cs)

C#

```
using Microsoft.AspNetCore.Mvc;
using StudentRecordPWA.Models;
using System.Collections.Generic;

namespace StudentRecordPWA.Controllers
{
   public class StudentsController : Controller
   {
      public IActionResult Index()
      {
         List<Student> students = new List<Student>()
         {
            new Student { Id = 1, Name = "Rahul", Course = "BSc IT", Age = 20 },
            new Student { Id = 2, Name = "Amit", Course = "BCA", Age = 21 },
            new Student { Id = 3, Name = "Sneha", Course = "BSc CS", Age = 19 }
         };
         return View(students);
      }
   }
}
```

Index View (Views/Students/Index.cshtml)

HTML

```
@model IEnumerable<StudentRecordPWA.Models.Student>

<h2>Student Records</h2>

<table border="1" cellpadding="10">
   <tr>
      <th>Id</th>
      <th>Name</th>
```

```
        <th>Course</th>
        <th>Age</th>
    </tr>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.Name</td>
            <td>@item.Course</td>
            <td>@item.Age</td>
        </tr>
    }
</table>
```

Manifest File (wwwroot/manifest.json)

JSON

```
{
  "name": "Student Record PWA",
  "short_name": "StudentPWA",
  "start_url": "/Students",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Service Worker File (wwwroot/service-worker.js)

JavaScript

```
const CACHE_NAME = "student-pwa-cache";
const urlsToCache = [
    "/Students"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});
```

```
self.addEventListener("fetch", event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

Manifest Link Tag (_Layout.cshtml - inside <head>)

HTML

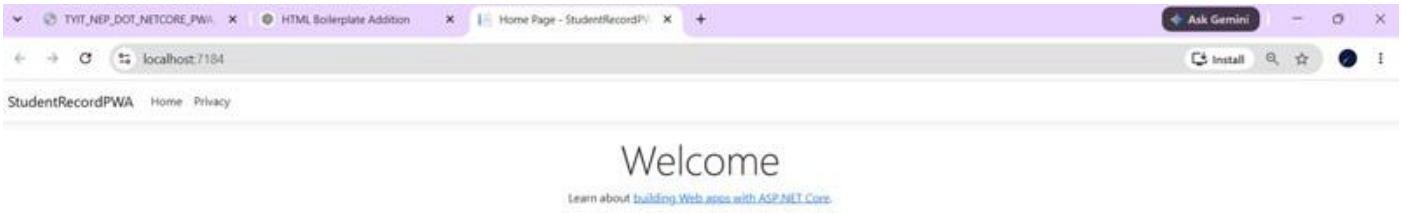
```
<link rel="manifest" href="/manifest.json">
```

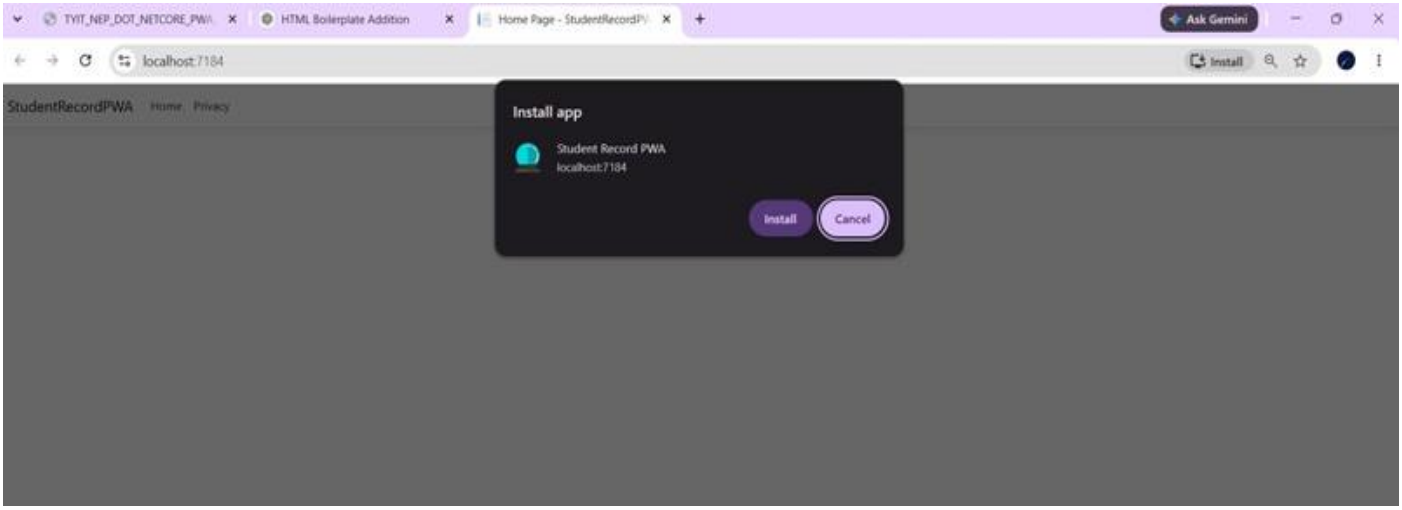
Service Worker Registration Script (_Layout.cshtml - before </body>)

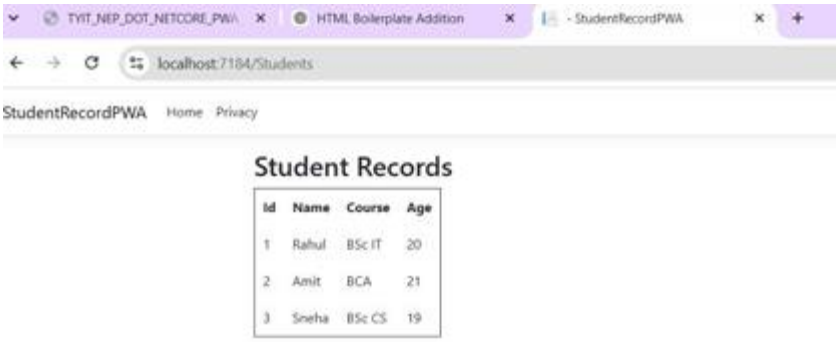
HTML

```
<script>
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/service-worker.js')
            .then(() => console.log('Service Worker Registered'));
    }
</script>
```

OUTPUT:






TYIT_NEP_DOT_NETCORE_PWA
HTML Boilerplate Addition
- StudentRecordPWA
localhost:7184/Students
StudentRecordPWA
Home
Privacy
Student Records
Id
Name
Course
Age
1
Rahul
BSc IT
20
2
Amit
BCA
21
3
Sneha
BSc CS
19

PRACTICAL 8A:

AIM: Create a PWA Application with an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json.

PROGRAM CODE :

Program.cs

C#

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

HTML Page (wwwroot/index.html)

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Basic PWA</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="manifest" href="manifest.json">
</head>
<body>
    <h1>My Progressive Web Application</h1>
    <p>This is the starting page of PWA.</p>
</body>
</html>
```

Manifest Link Tag (index.html - inside <head>)

HTML

```
<link rel="manifest" href="manifest.json">
```

Manifest File (wwwroot/manifest.json)

JSON

```
{
  "name": "Basic Progressive Web Application",
  "short_name": "BasicPWA",
  "description": "Simple PWA Example",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "orientation": "portrait",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
```

```
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

OUTPUT:



8B:

AIM: Create a simple Web API that returns student data in JSON format.

PROGRAM CODE :

Model File (Models/Student.cs)

C#

```
namespace StudentAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Course { get; set; }
        public int Age { get; set; }
    }
}
```

API Controller File (Controllers/StudentController.cs)

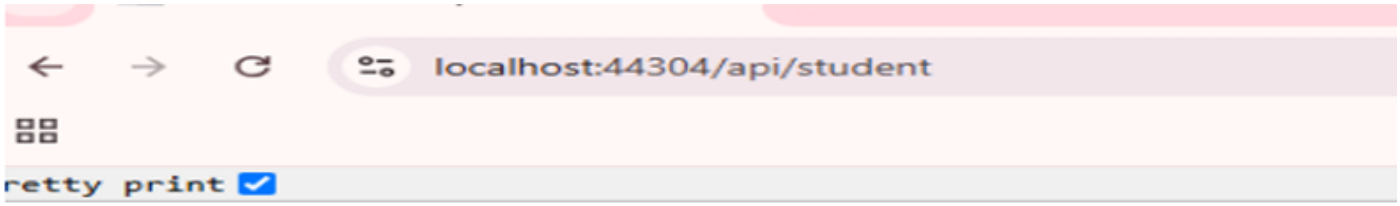
C#

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;
using System.Collections.Generic;

namespace StudentAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
```

```
        [HttpGet]
        public List<Student> GetStudents()
        {
            return new List<Student>()
            {
                new Student
                {
                    Id = 1,
                    Name = "Rahul",
                    Course = "BSc IT",
                    Age = 20
                },
                new Student
                {
                    Id = 2,
                    Name = "Sneha",
                    Course = "BCA",
                    Age = 21
                }
            };
        }
    }
}
```

OUTPUT:




```
{
    "id": 1,
    "name": "Rahul",
    "course": "BSc IT",
    "age": 20
},
{
    "id": 2,
    "name": "Sneha",
    "course": "BCA",
    "age": 21
}
```

PRACTICAL 9A:

AIM: Develop CRUD operations using ASP.NET Core Web API and Entity Framework core.

PROGRAM CODE :

Model File (Models/Student.cs)

C#

```
namespace StudentAPIWithEF.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
    }
}
```

DbContext File (Data/AppDbContext.cs)

C#

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }

        public DbSet<Student> Students { get; set; }
    }
}
```

Configuration File (appsettings.json)

JSON

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Program Configuration (Program.cs)

C#

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Data;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

API Controller File (Controllers/StudentsController.cs)

C#

```
using Microsoft.AspNetCore.Mvc;
using StudentAPIWithEF.Data;
using StudentAPIWithEF.Models;
using System.Linq;
namespace StudentAPIWithEF.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public StudentsController(AppDbContext context)
        {
            _context = context;
        }
        [HttpGet]
        public IActionResult GetStudents()
        {
            return Ok(_context.Students.ToList());
        }
        [HttpGet("{id}")]
        public IActionResult GetStudent(int id)
        {
            var student = _context.Students.Find(id);
            if (student == null) return NotFound();
            return Ok(student);
        }
        [HttpPost]
        public IActionResult AddStudent(Student student)
        {
            _context.Students.Add(student);
```

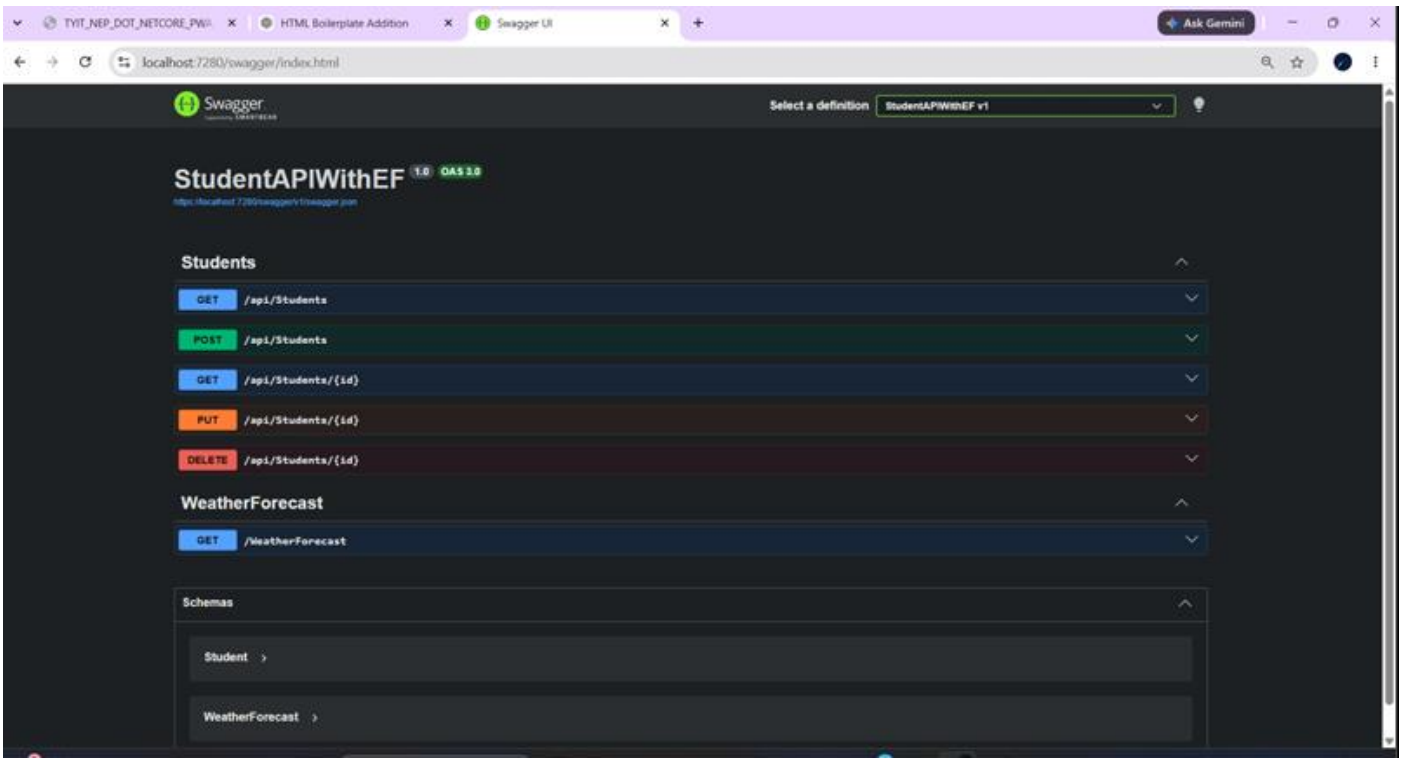
```
            _context.SaveChanges();
            return Ok(student);
        }
        [HttpPut("{id}")]
        public IActionResult UpdateStudent(int id, Student student)
        {
            var data = _context.Students.Find(id);
            if (data == null) return NotFound();

            data.Name = student.Name;
            data.Age = student.Age;
            data.Course = student.Course;

            _context.SaveChanges();
            return Ok(data);
        }
        [HttpDelete("{id}")]
        public IActionResult DeleteStudent(int id)
        {
            var student = _context.Students.Find(id);
            if (student == null) return NotFound();

            _context.Students.Remove(student);
            _context.SaveChanges();
            return Ok();
        }
    }
}
```

OUTPUT:



9B:

AIM: Create an API endpoint to calculate Factorial of a Number and test it using Postman.

PROGRAM CODE :

API Controller File (Controllers/MathController.cs)

C#

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult GetFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest("Factorial not possible for negative numbers.");
            }

            long factorial = 1;
            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }

            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
    }
}
```

OUTPUT:

